

CIT-321 / OS / MAM / 20 February 2025

Chapter 6: Synchronization Tools

- Concurrent process

Def: A Cooperating Process is ~~one~~ one that can affect or be affected by other processes executing in the ~~process~~ system.

→ can share a logical address space

logical address space (code, data) → shared memory

Concurrent access to shared data may result in ~~data~~ data inconsistency.

Synchronization tools that would enable data consistency.

* Chapter Objectives

Describe the critical-section problem and
illustrate a race condition
situation

Solving $\leftarrow$ using Hardware
compare and swap operation.

Demonstrate how Mutex, Bcu, semaphore,
monitors
~~locks~~

Producer-Consumer problem

#CIT-321/OS / MAM / 25 February 2025

Chapter - 6

Synchronization tool

G.1 Background

Page - 258

→ Producer - Consumer Code

⊗ race condition

↳

G.2 Critical-Section • Problem

Process

1. Mutual Exclusion

2. Progress

2. Bounded waiting

Segment
of code

⊗ entry section

critical section

Exit section
remainder section

before fig 6.2

#CIT-321 / OS / MAM / 27 February 2025

To solve critical section problem, we need to maintain

⇒ Two general approaches are used to handle critical section problem in operating system

1. Preemptive Kernel - A preemptive ~~kernel~~ kernel allows a process to be preempted while it is running in the kernel mode.

2. Non-preemptive kernel - Does not allow a process running in kernel mode to ~~be~~ be preempted.

→ Race condition free.

⇒ Peterson's Solution → software-based solution

↳ A classical software-based classical solution to the critical ~~section~~ section problem.

→ Provides a good alternative description

Demonstrate some complexity involved in designing software and address the requirement of ME, mutual exclusion, progress, and bounded waiting.

$P_i, P_j; j = 1 - i$

- requires two process to share two data item.

int turn;
boolean flag[2];
indicate ^{whose} turn it is to enter the critical section [^] if ~~the~~ turn == i, P_i allow ^{c.s.}

~~in 4 / turn;~~
~~boolean / flag [2];~~

flag - array is used to indicate if a process
that alternate execution between their
critical sections and remainder sections

#CIT-321 / OS / MAM / 28 February 2025

⇒ Peterson's Solution

fig 6.3

fig ~~6.3~~ 6.4

6.4 Hardware Support for Synchronization

6.4.1 Memory Barriers

P-339

6.4.2 Hardware Instructions

fig 6.5

next: compare and swap

#CIT-321 / OS / MAM / 01 March 2025

Compare and Swap
Hardware Instruction

Atomic Process

fig 6.7

fig 6.8

fig 6.9

6.5 Mutex locks

↳ mutual - exclusion

fig 6.10

Lock Contention

what is meant by "short duration."?

Semaphore

6.6

#CIT-321 / OS / MAM / 04 March 2025

⇒ 6.7 Monitors

Semaphores

$\text{mutex} = 1$

$\text{wait}(\text{mutex}) \rightarrow \text{CS}$

$\downarrow$ $\text{signal}(\text{mutex}) \rightarrow \text{after}$

⇒ 6.7.1 Monitor usage

Abstract data type $\rightarrow$ ADT

function-based solution $\rightarrow$ Monitor

figure 6.11

$\hookrightarrow$ condition

$\hookrightarrow$

condition x, y ;

$x.\text{wait}()$ $\rightarrow$ suspended

$x.\text{signal}()$ $\rightarrow$ Resume

⇒ 6.7.2 Implementing a monitor using Semaphores

next = 0 suspend

next count

wait(mutex) $\rightarrow$ CS $\rightarrow$ signal(mutex) $\rightarrow$ After

How monitors are used for resolving — ?

$\Rightarrow$ 6.7.3 Resuming Processes within a Monitor

FCFS $\rightarrow$ First come, First serve.

$\hookrightarrow$ Priority Queue

def: Liveness

$\Rightarrow$ 6.8.1 Deadlock

$\Rightarrow$ 6.8.2 Priority

X169 Evaluation

Self
study

CIT-321 / OS / MAM / 10 March 2025

Chapter 7 : Synchronization Examples

Problems in 6th chapter

↳ Race condition

Three problems :

1. Bounded-Buffer

2. Readers-writers

3. Dining Philosopher

7-1.1 The Bounded-Buffer Problem

↳ shared data structure

```
int n;  
semaphore mutex = 1;  
semaphore empty = n;  
semaphore full = 0;
```


Fig: 7.1 ; Producer Process: $\Rightarrow$ full buffer

```
while (true) {
```

```
    ...
```

```
    /* produce an item in next produced to
```

```
    ...
```

```
    wait(empty);
```

```
    wait(mutex);
```

```
    ...
```

```
    /* add next produced to the buffer by
```

```
    signal(mutex);
```

```
    signal(full);
```

```
}
```

Fig. 7-2 Consumer Process: $\Rightarrow$ empty buffer.

while (true) {

wait (full);

wait (mutex);

...

/* remove an item from buffer
to next consumed */

...

signal (mutex);

signal (empty);

/* consume the item in next
consumed */

...

}

7.1.2 The Reader - Writer Problem

~~Reader-writer~~

↳ first reader-writer problem

↳ ~~No reader requires~~ that no reader be kept waiting unless a writer has already obtained permission to use the shared object.

↳ second readers-writer problem

↳ once a writer is ready, that writer performs its ~~writing~~ ~~writer~~ as soon as possible.

Shared Data structure:

Semaphore: $rw_mutex = 1;$

semaphore mutex = 1;

```
int read_count = 0;
```

(1901)

1. The first part of the document is a list of names and dates, which appears to be a record of some kind. The names are written in a cursive script, and the dates are in a more formal, printed style. The list is organized into two columns, with names on the left and dates on the right.

(Over the bar)

(Sittler - 1900) Page 12

• (no. 100) 100/100

Fig: 7.3 Writer Process

```
while (true) {  
    wait (rw-mutex);  
    ...  
    /* writing is performed */  
    ...  
    signal (rw-mutex);  
}
```

Fig: 7.4 Reader Process

```
while (true) {  
    wait (mutex);  
    read-count ++;  
    if (read-count == 1)  
        wait (rw-mutex);  
    signal (mutex);  
    /* reading is performed */  
    ...  
    wait (mutex);  
    read-count --;  
    if (read-count == 0)  
        signal (rw-mutex);  
    signal (mutex);  
}
```


7.1.3 The Dining Philosophers Problem

Fig: 7.6 The structure of philosopher:

```
while (true) {  
    wait(chopstick[i]);  
    wait(chopstick[(i+1) % 5]);  
    // ...  
}
```

Fig: 7.7 Implementation of Dining Philosophers

CIT-321 / OS / MAM / 17 March 2025

Chapter 8: Deadlocks

~~def:~~ $\hookrightarrow$ Process
 $\hookrightarrow$ Thread

~~Short def:~~ A process is a program in execution.

A thread is a basic unit of CPU utilization.

$\Rightarrow$ 8.1 System Model

$\Rightarrow$ 8.2 Deadlock in Multi-threaded Applications

$\Rightarrow$ 8.2.1 Linelock

$\Rightarrow$ 8.3 Deadlock Characterization

$\hookrightarrow$ 8.3.1 Necessary Conditions

- | | |
|---------------------|------------------|
| 1. Mutual Exclusion | 4. Circular wait |
| 2. Hold and wait | |
| 3. No preemption | |

8.3.2 Resource Allocation Graph

↳ System resource allocation graph.

$$T_i \rightarrow R_j$$

$$R_j \rightarrow T_k$$

$$V \rightarrow T, R$$

$$T = \{T_1, T_2, \dots, T_n\}$$

$$R = \{R_1, R_2, \dots, R_m\}$$

E

next: methods for handling deadlocks

#CIT-321 / OS / MAM /

8.4 methods for handling Deadlocks

8.5 Deadlock Prevention

⇒ 8.5.1 Mutual Exclusion

for a system to be deadlock ! mutual exclusion

⇒ 8.5.2 Hold and Wait

⇒ 8.5.3 No Preemption

⇒ 8.5.4 Circular wait

8.6 Deadlock Avoidance

- ↳ low device utilization
- ↳ reduced system throughput

↳ 1. Additional Information

↳ 2. Maximum no. of R type

⇒ 8.6.1 Safe State

↳ safe sequence

⇒ 8.6.2 Resource Allocation Graph Algorithm (One ~~star~~ instance of each resource)

↳ Claim edge } fig: 8.9, 8.10
↳ Assignment }

⇒ 8.6.3 Banker's Algorithm

↳ data structure

↳ Available

↳ Max

↳ Allocation

↳ Need

8.6.3.1 - Safety Algorithm

8.6.3.2

8.6.3.3 An Illustrative Example

CTS-321 / OS / MAM / 20 March 2025

8.7 Deadlock Detection

⇒ 8.7.1 Single Instances of Each Resource Type

↳ wait-for → Graph

↳ Resource Allocation Graph

⇒ 8.7.2 Several Instances of Resource Type

algorithm employed are similar to Bankers algorithm²

- Available
- Allocation

8.7.3 Detection - Algorithm using

self reading

8.8 Recovery from Deadlock

8.8.1 Process and Thread Termination

- Abort all deadlocked processes
-

P. 392 many factors

8.8.2 Resource Preemption

1. Selecting a victim
2. Roll back
3. Starvation

Chapter 9: Memory Management

Fig: 9.1

↳ base + limit

9.1. Address Binding

- Compile time

-

- Execution time

Fig: 9.4

MMU

9.1.3

Fig: 9.5

~~MMU~~

9.3 Paging

9.3.1 Basic method

Fig 9.8

Chapter 9: Memory Management

Fig: 9.1

↳ base + limit

9.1. Address Binding

- Compile time

-

- Execution time

Fig: 9.4
MMU

9.1.3

Fig: 9.5

~~MMU~~

9.3 Paging

9.3.1 Basic method

Fig 9.8